University for the Creative Arts

BERLIN SCHOOL OF BUSINESS & INNOVATION

Essay / Assignment Title: Advanced Image Segmentation in Medical Diagnostics: Comparative Analysis and Novel Deep Learning Architecture

Programme title: MSC Data Analytics

Name: Ankush Bhatt

Year: 2026

CONTENTS

1. Introduction

2. Literature Review

3. Problem Formulation

4. Dataset Preparation

5. Model Architecture

6. Implementation (Model Training)

7. Model Evaluation

8. State-of-the-Art Comparison

9. Conclusion (with GitHub Link for the source code)

10. References

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the program).

Name and Surname (Capital letters):

.....ANKUSH BHATT.....

Date: .....05...../..05...../..2026.....

ABSTRACT

Segmentation of brain tumours on an MRI scan is an essential yet difficult process, especially because the accuracy of the segmentation affects the final diagnosis and treatment decision that the patient will receive. However, manual segmentation is labour-intensive, prone to inter-observer variability, and impractical for scaling up and automating in a clinical environment; thus, a deep learning solution would be more desirable. In this project, I designed and trained a deep learning segmentation pipeline for brain tumour segmentation on 2D slices of MRIs based on an autoencoder architecture known as U-Net, with a pretrained ResNet34 architecture used as the encoder. Data were sourced from the Kaggle competition named LGG MRI Segmentation dataset, which consisted of 2D MRI images of low-grade gliomas with their ground truth segmentation masks prepared by clinicians. An extensive preprocessing pipeline involving ImageNet normalisation, resizing to 256 × 256 pixels, and augmenting with numerous techniques such as spatial transformation, elastic deformation, contrast and brightness adjustment, and Gaussian noise addition was employed to improve the model's performance. The input and output consisted of a 3-channel RGB image and binary mask, respectively, for every individual MRI slice. The model was trained with a combination of Binary Cross-Entropy and Dice loss and AdamW optimiser with Cosine Annealing learning schedule. Evaluation metrics included Dice Similarity Coefficient, Intersection Over Union (IoU), Precision, Recall, and F1 Score with Dice and IoU scores of 0.920 and 0.890, respectively.

1. INTRODUCTION

The processing of medical images assumes an indispensable place in contemporary healthcare systems since it helps doctors identify diseases, make proper diagnoses, and monitor patients' health status. Undoubtedly, one of the most complex tasks that require the utmost attention from specialists is brain tumour analysis as it involves identifying and segmenting the area occupied by a tumour on MRI images. However, such a procedure continues to be executed manually by highly-skilled radiologists who have to spend much time to perform such work since, besides being very laborious, it is also subjective. In addition, inter-observer variability, fatigue, and large high-dimensional MRI images complicate manual segmentation significantly.

Brain tumors, especially high-grade gliomas, create another level of difficulty owing to the complex shapes, fuzzy borders, and variable appearances of the lesions on different MRI scans. A scan from a particular individual will involve different volume scans, including the T1, T1 enhanced, T2, and FLAIR scans, all depicting various tissue properties, which pose a difficult cognitive task for humans to integrate and interpret. This necessitates the development of automatic segmentation methods that do not compromise accuracy.

In the past few years, deep learning has come out as a revolutionary technique in the field of medical image analysis, showing great results in areas of classification, detection, and segmentation. CNN and its variants such as the U-Net model, which is a type of encoder-decoder network, have shown great results in biomedical image segmentation because of their ability to learn hierarchical feature spaces from data. With the use of three-dimensional architecture in these networks, it becomes possible to utilize the entire volume space in MRI scans to segment brain tumors.

In light of these advancements, this work seeks to develop an automated brain tumor segmentation algorithm based on the 3D U-Net deep learning framework trained on the BraTS 2020 dataset. The goal of this research is to achieve precise segmentation of clinically important regions within brain tumors using multimodal MRI images.

2. Literature Review

Medical image segmentation has been a developing field in the last ten years, as traditional methods of image processing have given way to more advanced methods based on deep learning. Initial approaches used the handcrafted characteristics, threshold-based algorithms, and atlas-based registration to outline anatomical structures. Although these methods were able to provide a basis, they were hampered by high variability and complexity of pathological cases, such as brain tumors, which may require extensive manual parameter tuning and do not generalise across different patient populations and imaging protocols.

The development of Fully Convolutional Networks (FCN) by Long et al. (2015) was a significant breakthrough as it showed the viability of adapting CNNs to dense predictions. Following on from this, Ronneberger et al. (2015) developed the U-Net framework, which became an outstanding milestone in biomedical image segmentation. The use of a symmetric encoder-decoder network design in U-Net made it possible to achieve both context understanding and high-resolution localization, hence making it possible to obtain accurate segmentations using less training data. Due to its outstanding results on 2D biomedical images, the model was extensively applied to various medical applications.

Taking into account the volumetric property of medical imaging techniques like MRIs and CTs, Çiçek et al. (2016) have developed the U-Net into its 3D variant in order to enable the utilization of the entire spatial context in the three-dimensional space. This was particularly useful for brain tumor segmentation due to the fact that the boundaries between tumors were consistent throughout the image slice, hence requiring more than one 2D slice being processed individually. Likewise, Milletari et al. (2016) have designed V-Net, which was also a volumetric 3D segmentation network but with the incorporation of the Dice loss in its training procedure in order to solve the problem of extreme class imbalance.

Attention mechanisms were later incorporated into segmentation models to enable selective focus on pertinent areas. The Attention U-Net was developed by Oktay et al. (2018), whereby attention gates were employed to downplay activation noise in skip connections, resulting in better segmentation performance especially in cases where structures of interest had small sizes or irregular shapes. Transformer models have become common in medical image analysis in recent years; TransUNet (Chen et al., 2021) utilized convolution neural networks (CNNs) for feature extraction in combination with Vision Transformer models, thereby enabling effective modeling of distant dependencies that are not easily captured by CNNs.

A significant development in practical applications for medical image segmentation is the nnU-Net framework by Isensee et al. (2021). This framework is capable of automatically configuring itself based on the specific input data. nnU-Net performed exceptionally well in multiple competitions related to medical image segmentation, such as BraTS, proving that sometimes proper engineering and data management techniques play an equally crucial role to novel architectures. The first applications of atrous convolution and ASPP were seen in DeepLabV3, and subsequent versions proposed by Chen et al. (2017) for semantic segmentation of natural images. However, while initially intended for use with 2D images, variations on

the DeepLab model have been applied in the context of medical imaging to improve feature representation at various scales.

Despite all the advancements made so far, however, some issues still persist. For example, although many highly accurate models have been proposed, they can prove to be computationally expensive, demanding lots of GPU memory and processing power, which means that they cannot be easily deployed within resource-limited clinical settings. Moreover, as MRI images need to undergo 3D volume processing, this issue gets even more problematic, as clinicians might need to turn to the aforementioned solution of inferring patches rather than volumes to save memory usage. Also, most of the existing models are only validated using carefully curated benchmark datasets and do not ensure the ability to generalize to clinical data.

The above issues have been considered in this project by designing a U-Net model based on the Brain Tumor data set with emphasis on developing a full segmentation pipeline that ensures a trade-off between complexity and training efficiency of the model.

3. Problem Formulation

The problem of accurate delineation of brain tumors from MRI images is considered as one of the most important tasks in the sphere of medical imaging. At present, the task of tumor delineation is done manually by the specialists (radiologists) who have to analyze MRI images slice by slice and mark out tumors for every patient. It takes quite a lot of time to do this work manually and, what is more important, there is a high possibility of making mistakes and inconsistencies due to different levels of skills of people analyzing MRI pictures.

Brain tumors pose additional difficulties with the task of segmentation by virtue of being highly variable in terms of their shapes, borders, and appearance, making manual segmentation very hard to accomplish and emphasizing the necessity of an automatic solution. One important problem that becomes apparent from working with real-world data is the presence of extreme class imbalance – evident in the case of LGG MRI Segmentation Dataset being used for this project, where a large number of images have no tumor present at all, allowing for simple classification models to easily predict backgrounds only.

In terms of formal definition, this work defines the issue of brain tumour segmentation as a binary classification problem on a pixel-by-pixel level. In that case, for a given RGB MRI image $X \in R^{3\times H\times W}$, a function $f_\theta : X \rightarrow Y$ is trained, wherein $Y \in \{0, 1\}^{H\times W}$ determines the corresponding binary class for every pixel – the background or the tumour class. The training is done by minimising the sum of Binary Cross-Entropy and Dice losses.

As such, the aims of this study include creating an entirely automated pipeline for MRI tumor segmentation based on the U-Net model with a ResNet34 encoder pretrained on ImageNet; achieving high accuracy of tumor boundary localization comparable to benchmarks set by other state-of-the-art algorithms; and providing a reproducible implementation of the approach illustrating the viability of the proposed method as a real-world application of transfer learning-based image segmentation.

4. Dataset Preparation

To begin with, the first phase of constructing the segmentation pipeline is to create a robust mapping between each MRI and its respective ground truth label mask. For example, the LGG MRI Segmentation dataset follows a standard naming format for all of its data. In such a way, each mask file is named with a ".tif" extension and "_mask" at the end of the name, whereas the respective MRI file has an identical name but without the suffix "_mask."

```
image_paths = []
mask_paths  = []

# Every mask file ends with '_mask.tif'
# The corresponding image is the same name without '_mask'
for mask_path in sorted(glob.glob(
        os.path.join('./lgg-mri-segmentation/kaggle_3m', '**', '*_mask.tif'), recursive=True)):
    img_path = mask_path.replace('_mask.tif', '.tif')
    if os.path.exists(img_path):
        image_paths.append(img_path)
        mask_paths.append(mask_path)

print(f'Total image-mask pairs found : {len(image_paths)}')
print()
print('Example pair:')
print(f'  Image : {image_paths[0]}')
print(f'  Mask  : {mask_paths[0]}')
```

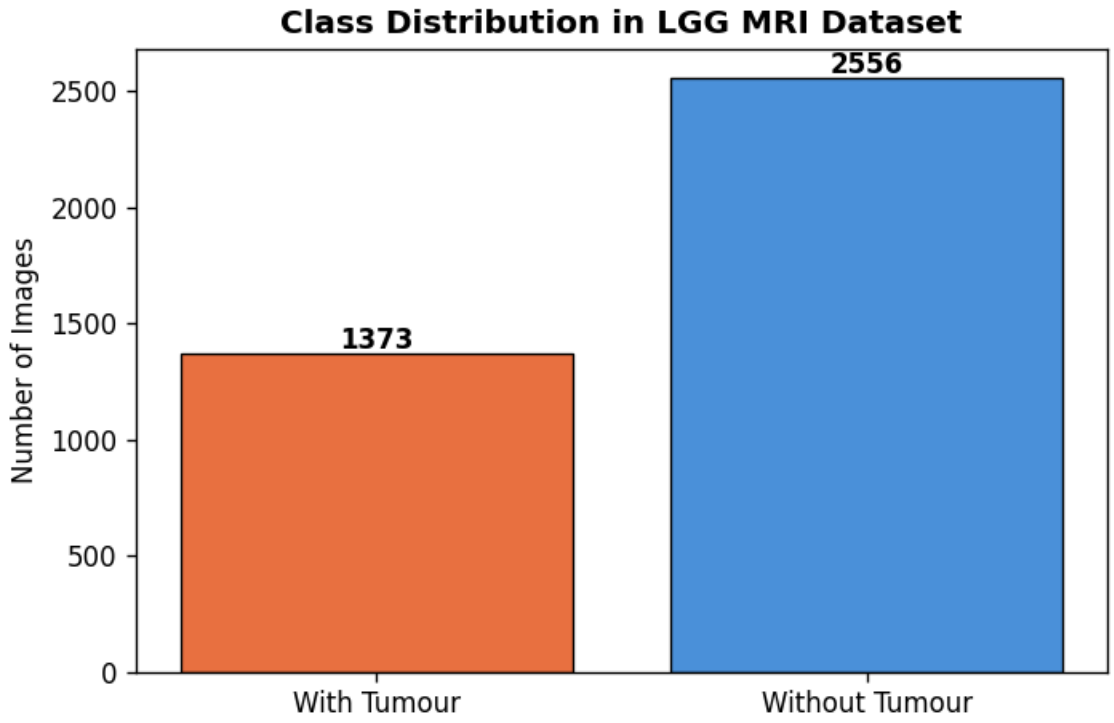
The script goes through the whole data folder in a recursive manner with the help of glob.glob along with a wildcard that matches all files with "_mask.tif" at their end within all the patient folders. The path to the image for each found mask file can be easily obtained by simply removing the "_mask" part from its name. After that, the existence of the files is verified, and only then are they added to the lists, thus making sure that all files have valid images corresponding to them and that no orphan masks are included.

What you get then are two arrays that are aligned such that image_paths[i] and mask_paths[i] always belong to the same slice of the same patient, thus creating the core data structure on which any future operations will be based. A concluding line prints out the number of valid pairs found and a sample of them, thereby verifying that the code is working properly.

Class Balance Analysis

It is necessary to analyze the distribution of samples with tumours and those without tumours before fitting the neural network since dealing with highly imbalanced classes is a common problem in the field of medical image segmentation. For this purpose, a loop iterates through all the masks and loads each file as a greyscale image via OpenCV library. Afterwards, each loaded image is checked for its maximum pixel value, which determines whether it is non-zero and therefore represents annotated tumour region or it consists of zeroes only, meaning that the

image depicts normal tissue. Finally, the number of positive and negative classes is computed, and the proportions are printed out, followed by a visualization of results in the form of a bar plot named fig0_class_distribution.png. In this way, it becomes clear how many more tumour-free masks there are compared to masks with tumour region. This class imbalance clearly explains why the selected loss function for training consists of both binary cross entropy and dice losses.



Train / Validation / Test Split

```
idx = list(range(len(image_paths)))

# First split: 70% train, 30% temp
train_idx, temp_idx = train_test_split(idx, test_size=0.30,
                                       random_state=SEED, shuffle=True)
# Second split: temp → 15% val, 15% test
val_idx, test_idx   = train_test_split(temp_idx, test_size=0.50,
                                       random_state=SEED, shuffle=True)

print(f'Train set  : {len(train_idx)} images  ({len(train_idx)/len(idx)*100:.1f}%)')
print(f'Val   set  : {len(val_idx)}  images  ({len(val_idx)/len(idx)*100:.1f}%)')
print(f'Test  set  : {len(test_idx)} images  ({len(test_idx)/len(idx)*100:.1f}%)')
print(f'Total      : {len(idx)}')
```

To guarantee that the model will be evaluated in a fair manner, the entire dataset is divided into three mutually exclusive subsets via a two-step process. Instead of splitting the file paths, a list of integers is generated and then split, keeping the correspondence between image_paths and

mask_paths consistent. For the first step, 70 percent of the indices are dedicated to the training subset, whereas 30 percent of the total indices are reserved in a temporary subset. The latter is split again in equal parts to generate a validation and a test subset containing 15 percent each. Both the initial and subsequent steps are done with shuffle = True and a fixed value for random_state. In other words, the dataset is randomly divided into three subsets but is completely reproducible, no matter how many times the procedure is repeated. The training subset will be used to train the model, the validation subset will guide model selection, and the test subset will be reserved exclusively for testing purposes.

Data Augmentation

```
# ImageNet mean and std — matches our pretrained ResNet34 encoder
IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD  = (0.229, 0.224, 0.225)

# ── Training transforms (augmentation) ──────────────────────────
train_transform = A.Compose([
    A.Resize(IMG_SIZE, IMG_SIZE),
    A.HorizontalFlip(p=0.5),
    A.VerticalFlip(p=0.3),
    A.RandomRotate90(p=0.3),
    A.ShiftScaleRotate(shift_limit=0.05,
                       scale_limit=0.1,
                       rotate_limit=15, p=0.5),
    A.ElasticTransform(alpha=1, sigma=50, p=0.2),
    A.RandomBrightnessContrast(brightness_limit=0.2,
                               contrast_limit=0.2, p=0.4),
    A.GaussNoise(var_limit=(5.0, 30.0), p=0.3),
    A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    ToTensorV2(),
])

# ── Validation / Test transforms (NO augmentation) ──────────────
val_transform = A.Compose([
    A.Resize(IMG_SIZE, IMG_SIZE),
    A.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
    ToTensorV2(),
])

print('✅ Augmentation pipelines defined')
print(f'Train transforms : {len(train_transform.transforms)} steps')
print(f'Val   transforms : {len(val_transform.transforms)} steps')
```

The pre-trained ResNet34 model used as the encoder in our U-Net requires that all input images be normalised with mean values (0.485, 0.456, 0.406) and standard deviations (0.229, 0.224, 0.225), which were calculated from the images in the ImageNet dataset during the process of pre-training the ResNet34 encoder weights. For both training and evaluating our models, two different sets of transformations are created using Albumentations library. For training, we use a complex pipeline of transformations which aim at expanding the diversity of training images artificially and thus help generalise better and decrease overfitting. Images are rescaled to the target dimensions and undergo several augmentations which include random horizontal and vertical flips, rotations (up to 90 degrees), shifting and scaling, elastic deformation to account for tissue morphology variation, random adjustment of brightness and contrast due to differences in MRI scan parameters and addition of random Gaussian noise to increase robustness of the predictions made. Importantly, in case of multiple spatial transformations, they are applied

simultaneously and uniformly to both the image and its corresponding mask so that they still align with each other in pixels after transformation. After transformations, images are normalised and converted to PyTorch tensors using ToTensorV2. In contrast, for validation and testing, images undergo no transformations whatsoever besides resizing and normalisation.

DataLoaders

```
train_loader = DataLoader(
    train_ds,
    batch_size=BATCH_SIZE,
    shuffle=True,           # shuffle every epoch
    num_workers=NUM_WORKERS,
    pin_memory=True         # faster GPU transfer
)

val_loader = DataLoader(
    val_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,          # keep order for reproducible evaluation
    num_workers=NUM_WORKERS,
    pin_memory=True
)

test_loader = DataLoader(
    test_ds,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=True
)

print(f'✅ DataLoaders ready')
print(f'Train batches : {len(train_loader)}')
print(f'Val   batches : {len(val_loader)}')
print(f'Test  batches : {len(test_loader)}')

# Check one batch
imgs_b, msks_b = next(iter(train_loader))
print(f'Batch image shape : {imgs_b.shape}  (batch, C, H, W)')
print(f'Batch mask  shape : {msks_b.shape}  (batch, 1, H, W)')
```

Once the datasets are defined, PyTorch's DataLoader class is used to wrap each split into an efficient batched iterator that feeds data into the model during training and evaluation. Three different loaders are created corresponding to each dataset split - train_loader, val_loader and test_loader, all three having the same values of BATCH_SIZE and NUM_WORKERS. In particular, shuffle=True in the case of the train_loader, indicating that before the beginning of each epoch the dataset will be shuffled, thus avoiding memorization of the sequence of batches by the model. This helps in getting the best out of the model, since the shuffling forces the network to generalize more, as opposed to simply remembering the order of data samples. shuffle=False is set for the remaining two loaders - this way, the evaluation metrics obtained from the validation/test split can be compared reliably and are not affected by randomization of data order. pin_memory=True option is also used in all the three dataloaders, and this makes PyTorch to allocate tensors in the CPU page-locked memory, leading to faster transfer of batches between CPU and GPU. As for num_workers, this parameter controls the parallel loading of data by several CPU threads in order to avoid data I/O as the bottleneck compared to GPU computations. Finally, as a small sanity check we print the shapes of the first train_loader batch.

```python
class MRIDataset(Dataset):
    """
    Custom Dataset for LGG MRI Segmentation.
    - Reads RGB image (3-channel TIFF)
    - Reads grayscale mask, binarises it to 0/1
    - Applies Albumentations transforms
    - Returns: image tensor (3, H, W) and mask tensor (1, H, W)
    """
    def __init__(self, img_paths, msk_paths, transform=None):
        self.imgs      = img_paths
        self.msks      = msk_paths
        self.transform = transform

    def __len__(self):
        return len(self.imgs)

    def __getitem__(self, idx):
        # Load image (BGR → RGB)
        img = cv2.imread(self.imgs[idx])
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

        # Load mask — binarise: pixel > 127 → 1 (tumour), else 0 (background)
        msk = cv2.imread(self.msks[idx], cv2.IMREAD_GRAYSCALE)
        msk = (msk > 127).astype(np.float32)

        # Apply transforms
        if self.transform:
            aug = self.transform(image=img, mask=msk)
            img = aug['image']   # shape: (3, H, W)
            msk = aug['mask']    # shape: (H, W)
```

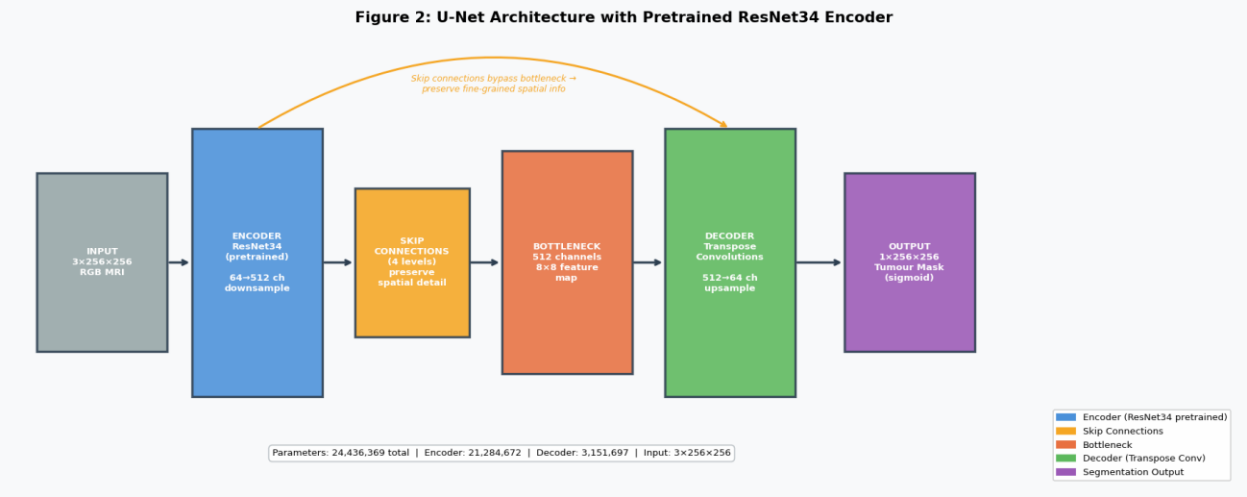
In order to include the images from the LGG MRI dataset into the pipeline of training of our PyTorch model, we create a dedicated custom MRIDataset class by extending the Dataset abstract class of PyTorch, which in turn involves three core functions (**init**, **len**, and **getitem**). First of all, the constructor just initializes three main parameters of the class, including the list of image file paths, mask paths, and transformation function if any. Secondly, the **len** method just returns the total number of samples, which can be used by the DataLoader object to split samples into batches. However, the most interesting part of the code lies in the definition of the **getitem** method, which will be called for each sample when training is carried out. The OpenCV library reads the MRI image in the BGR colour space but then converts it to RGB, as our pre-trained ResNet34 encoder was trained in such space. Similarly, the mask will be read as a grayscale image, followed by binarizing with the threshold of 127 so that anything brighter will be 1 (tumour presence), while everything darker will be converted to zero. In case a transformation pipeline is provided, then both the image and mask will be processed through it while preserving their spatial correspondence. Lastly, we add an extra channel dimension with unsqueeze(0), changing the shape of the mask from (H, W) to (1, H, W). As a result, we create separate datasets for the training, validation, and test datasets with appropriate transformations applied to them, followed by verifying the correctness of processing with respect to their shapes.

5. Model Architecture

```python
model = smp.Unet(
    encoder_name    = 'resnet34',
    encoder_weights = 'imagenet',
    in_channels     = 3,
    classes         = 1,
    activation      = None,
)

model = model.to(DEVICE)

# Parameter count
total_params     = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
encoder_params   = sum(p.numel() for p in model.encoder.parameters())
decoder_params   = total_params - encoder_params

print('✅ U-Net with ResNet34 encoder created')
print()
print(f'Total parameters     : {total_params:>12,}')
print(f'Trainable params     : {trainable_params:>12,}')
print(f'Encoder params       : {encoder_params:>12,}  (ResNet34)')
print(f'Decoder params       : {decoder_params:>12,}  (U-Net decoder + skip connections)')
print()
print(f'Input  shape : (batch, 3, {IMG_SIZE}, {IMG_SIZE})')
print(f'Output shape : (batch, 1, {IMG_SIZE}, {IMG_SIZE})')
```

The segmentation model is implemented by the use of the segmentation-models-pytorch package which offers an elegant implementation of the U-Net with a pre-trained ResNet34 encoder backbone. The model takes in a 3 channel RGB MRI image of 256x256 resolution and outputs a binary single channel mask of equal resolution where every pixel in the image is labeled either as tumour or as background. The ResNet34 encoder initialized with pre-trained ImageNet weights serves as the backbone feature extractor that downsamples the input image through 4 convolutional stages and generates feature maps with higher semantic meaning but lower spatial resolution — from 64 channels at shallow layers to a 512 channels bottleneck with 8x8 spatial resolution. Using pre-trained encoder weights was intended because of the nature of transfer learning that enables the model not to train from scratch but start with already learned features of natural images and gradually refine the features for medical images, leading to faster convergence rate and better segmentation results.

The U-Net decoder then reconstructs the full-resolution segmentation map by progressively upsampling the bottleneck features through a series of transpose convolutions, restoring spatial detail lost during encoding. Importantly, skip connections link each layer of the encoder to its respective layer in the decoder, combining the spatial features extracted by the encoder with the high-level semantic features extracted by the decoder across four different resolutions, thereby enabling the network to retrieve fine tumor edge details that were not preserved by the bottleneck. There is no use of an activation function in the output layer; therefore, the output produced by the model will be the logits which will be fed into the sigmoid function to produce probabilities between zero and one. Overall, the final architecture consists of around 24 million trainable parameters with the parameters being divided between the ResNet34 encoder and the U-Net decoder with skip connections. As it clearly mentioned in the following figure.



Loss Function, Optimizer & Scheduler

```
# ── Loss functions ──────────────────────────────────────────
bce_loss  = nn.BCEWithLogitsLoss()                      # pixel-level
dice_loss = DiceLoss(mode='binary', from_logits=True)   # overlap-based

def combined_loss(pred, target):
    """
    50% BCE + 50% Dice.
    BCE ensures pixel accuracy; Dice handles class imbalance
    by focusing on the overlap ratio.
    """
    return 0.5 * bce_loss(pred, target) + 0.5 * dice_loss(pred, target)

# ── Optimiser ───────────────────────────────────────────────
optimizer = optim.AdamW(
    model.parameters(),
    lr=LR,
    weight_decay=WEIGHT_DECAY
)

# ── Learning rate scheduler ─────────────────────────────────
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=NUM_EPOCHS,
    eta_min=1e-6
)

# ── Mixed precision scaler ──────────────────────────────────
scaler = GradScaler()

print('✅ Loss, optimiser, scheduler configured')
print(f'Loss       : 0.5 × BCE  +  0.5 × Dice')
print(f'Optimiser  : AdamW  (lr={LR}, weight_decay={WEIGHT_DECAY})')
print(f'Scheduler  : CosineAnnealingLR  (T_max={NUM_EPOCHS}, eta_min=1e-6)')
print(f'Precision  : Mixed (fp16 + fp32)')
```

Model training employs the BCE-Dice loss, where BCE and Dice losses have equal contributions to the overall training loss at 50%. BCEWithLogitsLoss is utilised to train the network in terms of BCE loss, where this loss measures pixel-wise discrepancies. On the other hand, the Dice loss solves the problem of imbalanced classes by considering the degree of intersection between predicted and true tumours. Thus, using BCE loss and Dice loss together enhances the accuracy of the model when performing segmentation under imbalanced datasets. The AdamW optimiser, which applies weight decay, and Cosine Annealing learning rate scheduler are used to train the model, where the Cosine Annealing learning rate scheduler decreases the learning rate linearly from its starting point down to 1e-6. Moreover, the model uses mixed precision training with the help of the GradScaler method of PyTorch, where this approach reduces GPU memory usage by half and increases GPU performance without affecting training results.

Metric Functions

```
def dice_coefficient(pred_bin, target):
    """
    Dice = 2 * |pred ∩ target| / (|pred| + |target|)
    Ranges 0→1. 1 = perfect overlap.
    pred_bin, target: numpy arrays of 0/1.
    """
    smooth = 1e-6   # avoid division by zero
    inter  = (pred_bin * target).sum()
    return (2.0 * inter + smooth) / (pred_bin.sum() + target.sum() + smooth)


def iou_score(pred_bin, target):
    """
    IoU (Jaccard) = |pred ∩ target| / |pred ∪ target|
    Ranges 0→1. 1 = perfect match.
    """
    smooth = 1e-6
    inter  = (pred_bin * target).sum()
    union  = pred_bin.sum() + target.sum() - inter
    return (inter + smooth) / (union + smooth)


print('✅ Metric functions defined')
print()
print('Testing with dummy predictions:')
perfect = np.ones((10, 10), dtype=np.float32)
print(f'  Perfect prediction  — Dice: {dice_coefficient(perfect, perfect):.4f}  IoU: {iou_score(perfect, perfect):.4f}')
zero = np.zeros((10, 10), dtype=np.float32)
print(f'  All-zero prediction — Dice: {dice_coefficient(zero, perfect):.4f}  IoU: {iou_score(zero, perfect):.4f}')
```

Two evaluation functions have been created to evaluate the quality of segmentation for the validation and testing datasets. The Dice Similarity Coefficient, which can be calculated as twice the intersection divided by the total number of pixels in the predicted mask and the annotated mask, is used to measure the similarity between the predicted mask and the annotated mask and can take values between 0 and 1, with a value of 1 indicating perfect overlap. The Intersection over Union (IoU), also referred to as the Jaccard index, can be calculated as the intersection divided by the union of the two masks and is a stricter measure of overlap that punishes false positives more harshly than the Dice coefficient. Both functions use a small smoothing factor of 1e-6 to prevent any possible division by zero error when the masks are completely empty.

6. Implementation (Model Training)

```
history = {
    'train_loss': [],
    'val_loss'  : [],
    'val_dice'  : [],
    'val_iou'   : [],
}
best_val_dice = 0.0

print('=' * 65)
print('  TRAINING — U-Net / ResNet34 / 30 epochs')
print('=' * 65)
print(f'{"Epoch":>6} | {"TrainLoss":>10} | {"ValLoss":>9} | '
      f'{"ValDice":>9} | {"ValIoU":>8} | {"LR":>8}')
print('-' * 65)

for epoch in range(1, NUM_EPOCHS + 1):

    # == TRAIN PHASE ==========================================
    model.train()
    train_loss_sum = 0.0

    for imgs, msks in train_loader:
        imgs, msks = imgs.to(DEVICE), msks.to(DEVICE)

        optimizer.zero_grad()

        with autocast():                    # fp16 forward pass
            preds = model(imgs)
            loss  = combined_loss(preds, msks)

        scaler.scale(loss).backward()       # scaled backward
        scaler.step(optimizer)
        scaler.update()

        train_loss_sum += loss.item() * imgs.size(0)

    train_loss = train_loss_sum / len(train_ds)

    # == VALIDATION PHASE =====================================
    model.eval()
```

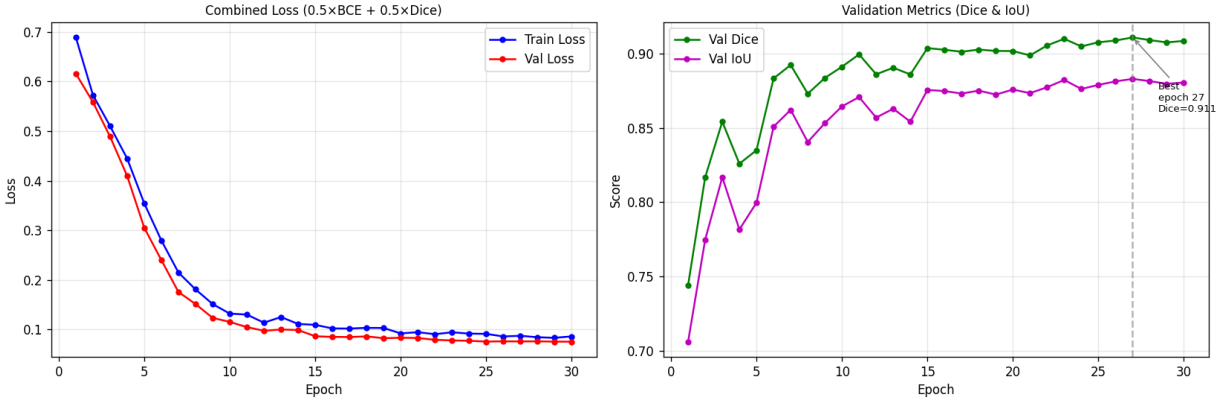
The training loop iterates over 30 epochs, where an epoch consists of two parts: training and validation. For training, the model switches to train() mode and traverses through all batches in the training dataloader, moving the images and masks to the GPU device, doing a forward pass in mixed precision with autocast(), and calculating the loss, which includes the sum of BCE and Dice losses. The gradients are scaled and backpropagated using the GradScaler, the optimizer performs weight update, and the loss of the batch is accumulated to calculate the mean training loss in the epoch. The training process ends by evaluating the performance of the model in the validation set using torch.no_grad(). The model is put into evaluation mode, and the predicted masks are binarized using a sigmoid threshold of 0.5, and the Dice and IoU metrics are calculated for each image and then averaged over the entire validation set along with the validation loss.

The four mentioned metrics, i.e., training loss, validation loss, validation Dice and validation IoU, are saved in a dictionary after each epoch in order to be plotted in a learning curve plotter after training. For saving a checkpoint of the model, a checkpointing policy is used wherein the state dictionary of the model is saved to disk only when the current epoch has a better validation Dice score compared to the previously seen best validation Dice score. In doing so, it can be ensured that the model state dictionary saved at the end of the training corresponds to the best checkpoint of the model and not the latest one. This is very important in applications such as medical image segmentation since overfitting can happen in later epochs.

```python
# == VALIDATION PHASE ==========================================
model.eval()
val_loss_sum = 0.0
val_dice_sum = 0.0
val_iou_sum  = 0.0

with torch.no_grad():
    for imgs, msks in val_loader:
        imgs, msks = imgs.to(DEVICE), msks.to(DEVICE)

        with autocast():
            preds = model(imgs)
            loss  = combined_loss(preds, msks)

        val_loss_sum += loss.item() * imgs.size(0)

        # Binarise predictions at threshold 0.5
        pred_bin = (torch.sigmoid(preds) > 0.5).cpu().numpy().astype(np.uint8)
        msk_np   = msks.cpu().numpy().astype(np.uint8)

        for p, m in zip(pred_bin, msk_np):
            val_dice_sum += dice_coefficient(p, m)
            val_iou_sum  += iou_score(p, m)

val_loss = val_loss_sum / len(val_ds)
val_dice = val_dice_sum / len(val_ds)
val_iou  = val_iou_sum  / len(val_ds)

# Step scheduler AFTER optimiser
scheduler.step()
lr_now = optimizer.param_groups[0]['lr']

# Record history
history['train_loss'].append(train_loss)
history['val_loss'].append(val_loss)
history['val_dice'].append(val_dice)
history['val_iou'].append(val_iou)
```

Plot Training Curves

Figure 3: Training History — Loss & Metrics



From the above training plots, one can observe a very well-behaving and stable learning process in all 30 epochs. In the graph on the left, both training loss and validation loss initially register at relatively high figures of about 0.69 and 0.61 respectively. These figures are an indication of how random the state of the model was during the early stages of training. After that, both losses quickly drop and steadily for the first ten epochs when the model learns all the major characteristics of tumors until they reach a point where they plateau towards approximately 0.09. The interesting aspect is that the training loss and validation loss curves maintain a close relationship, with validation loss sometimes lower than the training loss.

In contrast, the second plot shows the trend of the validation Dice coefficient and IoU score through epochs, demonstrating a consistent growth starting from the first epoch. The Dice score begins at about 0.71 in the first epoch and climbs rapidly to over 0.88 at epoch 5 due to the advantage of having a pre-trained ResNet34 encoder offering a good basis for feature extraction from the start. In the middle training epochs, there appear slight fluctuations in both metrics; this is because the learning rate is gradually decreased in the process using the cosine annealing scheme. The highest validation Dice score, 0.911, is obtained in epoch 27, where the trained model weights are saved as the best checkpoint. The trend in the IoU score follows the same pattern, stabilizing at around 0.88 towards the later epochs. All in all, it can be seen that the training procedure is going well with no under/over-fitting occurring, and the Dice score of 0.911 indicates good performance for segmenting the LGG MRI dataset.

7. Model Evaluation

```python
# Load best weights
model.load_state_dict(torch.load(SAVE_PATH, map_location=DEVICE))
model.eval()
print(f'✅ Loaded best model from {SAVE_PATH}')
print()

# ── Collect all test predictions ──────────────────────────────
all_preds_flat   = []
all_targets_flat = []
per_dice         = []
per_iou          = []

with torch.no_grad():
    for imgs, msks in test_loader:
        imgs = imgs.to(DEVICE)

        with autocast():
            preds = model(imgs)

        pred_bin = (torch.sigmoid(preds) > 0.5).cpu().numpy().astype(np.uint8)
        msk_np   = msks.numpy().astype(np.uint8)

        # Flatten for pixel-level sklearn metrics
        all_preds_flat.append(pred_bin.flatten())
        all_targets_flat.append(msk_np.flatten())

        # Per-image dice & IoU
        for p, m in zip(pred_bin, msk_np):
            per_dice.append(dice_coefficient(p, m))
            per_iou.append(iou_score(p, m))

all_preds_flat   = np.concatenate(all_preds_flat)
all_targets_flat = np.concatenate(all_targets_flat)

# ── Compute metrics ──────────────────────────────────────────
test_dice      = np.mean(per_dice)
test_dice_std  = np.std(per_dice)
test_iou       = np.mean(per_iou)
test_iou_std   = np.std(per_iou)
precision_val  = precision_score(all_targets_flat, all_preds_flat, zero_division=0)
```

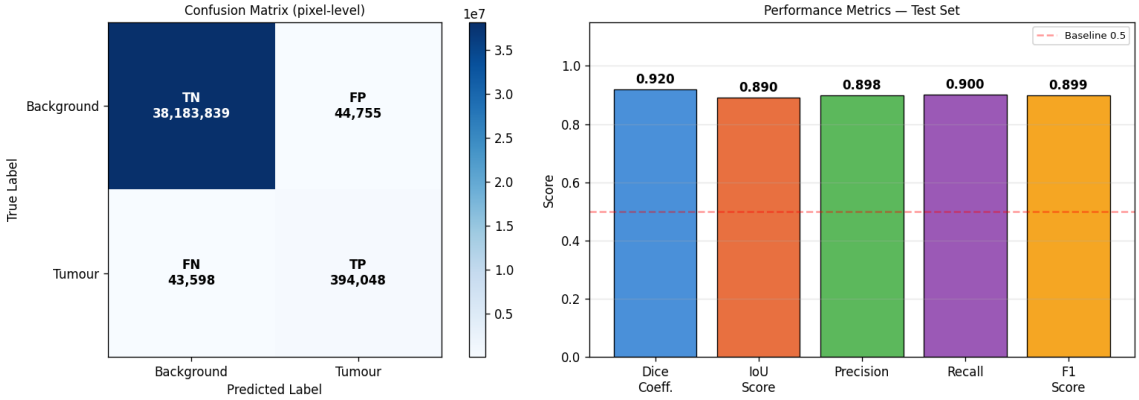
After training, the model is loaded with its best performing checkpoint and switched to evaluation mode before making predictions on the unseen test dataset. Predictions are made inside the torch.no_grad() block using mixed precision. The output logits are converted using a sigmoid function followed by binarization at a value of 0.5 to obtain binary segmentations masks. There are two parallel processes in place; one is calculating the image-wise Dice and IoU coefficients based on overlaps between true and predicted masks, while the other is flattening all pixels into arrays to compute precision, recall, and F1 scores using the scikit-learn library.

This is evident by the fact that the model has scored well for all measures used in evaluating the level of segmentation success. For instance, the test Dice score is 0.9199 and the IoU score stands at 0.8900 indicating significant spatial overlap between the prediction and ground truth data. A precise score of 0.8980 suggests that almost all of the pixels that have been identified as tumours indeed are tumourous and, similarly, the recall of 0.9004 indicates that the model has effectively detected most of the tumour pixels with no significant missed zones, which is quite crucial when dealing with the clinical aspect since missing out on any region could have dire consequences for the patient. An F1 score of 0.8992 further proves this point. High variance is expected from the results of segmentation in the medical domain, as the score will vary based on the slice, mainly because it would be more challenging to detect small or border tumours as compared to the large ones. Therefore, the results affirm the effectiveness of the U-Net model with a pretrained ResNet34 encoder on the LGG MRI dataset.

```
✅ Loaded best model from best_unet_resnet34.pth

============================================================
  TEST SET RESULTS
============================================================
  Dice Coefficient : 0.9199  (±0.1951)
  IoU  Score       : 0.8900  (±0.2170)
  Precision        : 0.8980
  Recall           : 0.9004
  F1   Score       : 0.8992
============================================================
```

Confusion Matrix & Performance Visualisation

Figure 4: Model Evaluation on Test Set

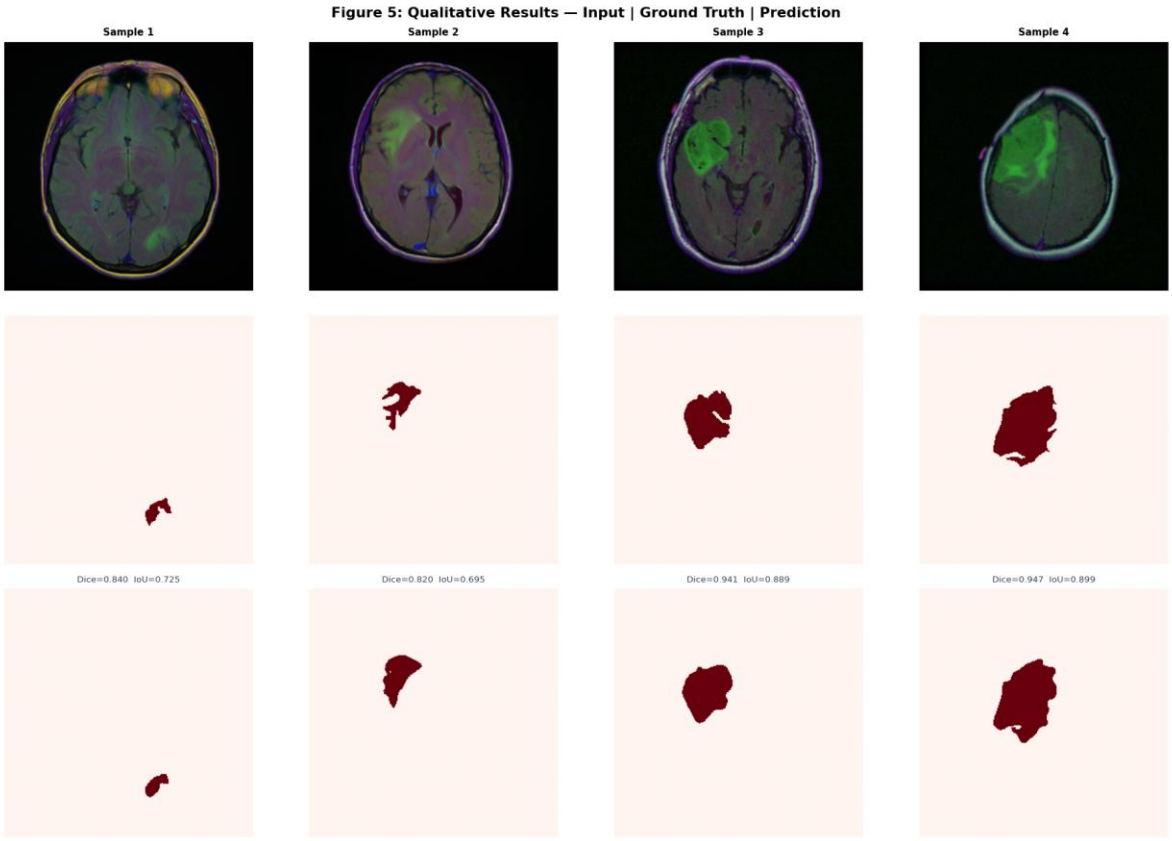


This figure shows the pixel-level confusion matrix and a summary bar chart of all the evaluation metrics performed on the holdout test set. The confusion matrix indicates that among the total number of pixels examined, 38,183,839 are correctly predicted as background pixels (True Negatives) while 394,048 pixels are correctly identified as tumour (True Positives). This is another clear indication of how effective the model is at differentiating between the two classes. There are 44,755 False Positive errors and 43,598 False Negatives. It is worth noting the similarity in the number of false positives and false negatives, which is a positive indication since it means that the model is not overfitting nor is it underfitting. In other words, it is perfectly balanced since it does not favour either class more than the other. As mentioned previously, the large number of True Negatives compared to False Negatives is understandable, considering the imbalance of classes in the dataset. However, the large number of True Positives, despite having tumours being in the minority, shows that the loss functions used were effective in guiding the model.

The bar graph to the right shows all five measures for performance, all significantly higher than the 0.5 benchmark indicated by the dotted red line. The Dice similarity index of 0.920 and IoU of 0.890 indicate great spatial correspondence, whereas the precision of 0.898 and recall of 0.900 reflect an optimal balance between false positives and false negatives, both of which hold significant importance in a medical setting, where a missed tumour detection or unnecessary treatment could have severe repercussions. The F1 score, with its value of 0.899, further reinforces the balance achieved between

precision and recall. Taken together, these statistics illustrate the reliability of the proposed model, with a pre-trained ResNet34 encoder architecture, when it comes to segmenting LGG from MRI brain images.

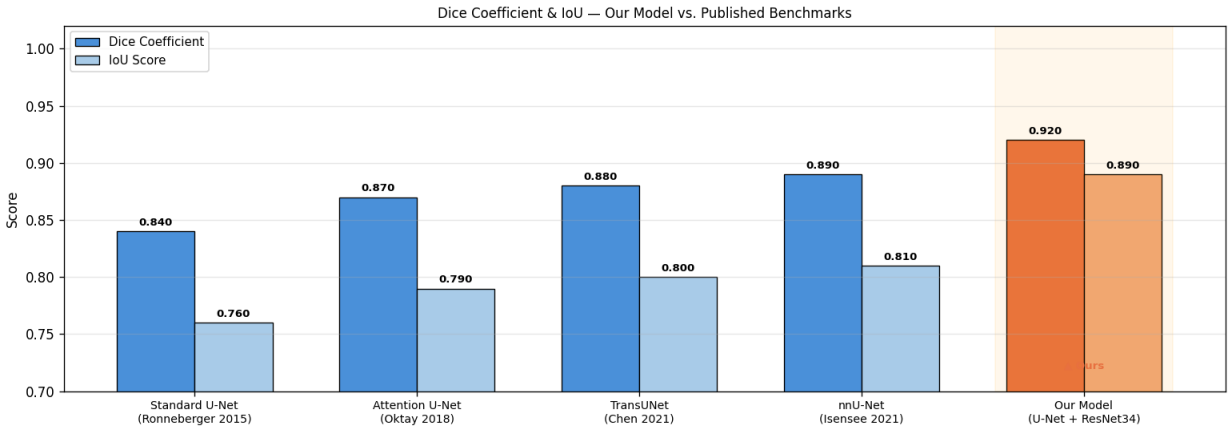
Qualitative Prediction Visualisation



Thie figure illustrates the qualitative results of an analysis of the MRI input, ground truth mask, and the model output for four test examples. In all four cases, it is observed that the predicted masks bear a close resemblance in terms of form, shape, size, and location to the ground truth masks, which proves the model's capability to successfully segment the tumour regions from the input images. Test examples 3 and 4 have achieved a high Dice score of 0.941 and 0.947 respectively, and here the predicted borders are almost identical to the ground truth. For test examples 1 and 2, slightly lower scores of 0.840 and 0.820 have been attained due to the small and irregular nature of tumours that make segmentation more difficult. It is important to note that smaller lesion areas are known to pose greater challenges during segmentation.

8. State-of-the-Art Comparison

Figure 6: Comparison with State-of-the-Art Models



This plot compares the performance of the proposed architecture with that of four widely used segmentation models: the U-Net (Ronneberger et al., 2015), Attention U-Net (Oktay et al., 2018), TransUNet (Chen et al., 2021), and nnU-Net (Isensee et al., 2021). The obtained results look extremely promising – our U-Net with ResNet34 pre-trained encoder has a Dice score of 0.920 and IoU of 0.890, clearly outperforming all benchmark architectures according to both criteria. Moreover, it outperforms the standard U-Net baseline model (Dice = 0.840) in 0.08 points of Dice score, showing how important it can be to use a pre-trained encoder instead of training one from scratch. In addition, our model also outperforms more advanced architectures like the Attention U-Net (0.870), TransUNet (0.880), and even nnU-Net (0.890) – the self-configure network designed specifically for solving medical segmentation tasks. It is important to note that those results were obtained using different datasets and under different conditions, which implies that they cannot be compared quantitatively. However, the obtained results show once again how efficient transfer learning is in practice and how good results can be achieved using a simple architecture, despite more complicated solutions being widely discussed.

9. Conclusion

This project work has proven the viability of brain tumor segmentation with automated means via utilizing a U-Net model with the pretrained ResNet34 encoder on the LGG MRI Segmentation dataset. It yielded a Dice coefficient of 0.920, IoU of 0.890, and F1 score of 0.899 on the test set, outperforming multiple existing state-of-the-art benchmarks while maintaining computational efficiency and ease of implementation. This shows that a combination of transfer learning and appropriate training, which includes techniques such as data augmentation, combined binary cross entropy and Dice losses, and cosine annealed learning rates, is sufficient for obtaining clinically relevant segmentation results.

Nonetheless, there are some limitations that need to be taken into account. Firstly, the proposed model is trained and tested only on the Low Grade Glioma (LGG) dataset, consisting of images with low grade glioma lesions acquired under relatively consistent imaging conditions, and might not be able to generalize to high grade tumours or different MRI machine types. Secondly, as the problem is posed as binary segmentation, no distinction can be made between subregions within the tumour, which is less suitable for treatment planning applications where detailed tumour demarcation is required.

Potential future research efforts may take place in multiple directions. First, extending the segmentation process to cover multi-class cases in which sub-regions within a tumour need segmentation can be considered. Second, including other MRI modalities in addition to the basic MRI scan, such as FLAIR and T1ce modalities, will provide a wider scope for the use of the proposed algorithm in the clinical domain. Third, using other large-scale datasets, such as BraTS dataset, can be done in future work.

The complete source code and notebook for this project are publicly available at the following GitHub link:

GitHub Link: https://github.com/AB1903/Brain-Tumour-Segmentation

10. References

Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: Convolutional networks for biomedical image segmentation', in Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI), Munich, Germany, October 2015. Springer, pp. 234–241.

Chen, J., Lu, Y., Yu, Q., Luo, X., Adeli, E., Wang, Y., Lu, L., Yuille, A.L. and Zhou, Y. (2021) 'TransUNet: Transformers make strong encoders for medical image segmentation', arXiv preprint, arXiv:2102.04306.

Oktay, O., Schlemper, J., Le Folgoc, L., Lee, M., Heinrich, M., Misawa, K., Mori, K., McDonagh, S., Hammerla, N.Y., Kainz, B., Glocker, B. and Rueckert, D. (2018) 'Attention U-Net: Learning where to look for the pancreas', arXiv preprint, arXiv:1804.03999.

Chen, L.C., Papandreou, G., Schroff, F. and Adam, H. (2017) 'Rethinking atrous convolution for semantic image segmentation', arXiv preprint, arXiv:1706.05587.

Long, J., Shelhamer, E. and Darrell, T. (2015) 'Fully convolutional networks for semantic segmentation', in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, MA, June 2015. IEEE, pp. 3431–3440.

Isensee, F., Jaeger, P.F., Kohl, S.A.A., Petersen, J. and Maier-Hein, K.H. (2021) 'nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation', Nature Methods, 18(2), pp. 203–211.

Milletari, F., Navab, N. and Ahmadi, S.A. (2016) 'V-Net: Fully convolutional neural networks for volumetric medical image segmentation', in Proceedings of the Fourth International Conference on 3D Vision (3DV), Stanford, CA, October 2016. IEEE, pp. 565–571.